

Simple Epidemic Simulator

Tero Kolari

2025-12-20

1. Introduction

This project implements a simple epidemic simulator on a two-dimensional discrete space. The population consists of `n_walk` identical walkers that move on a 2D grid of side length `box_len`. Each walker has an epidemiological state (healthy, sick, immune). The simulation is run for `sim_len` time steps. At each step, walkers (1) move randomly, (2) can infect healthy walkers with probability `p_sick`, if they are themselves sick, and (3) may heal (become immune) with probability `p_heal`. One motivation for this model is to study how simple interaction rules (random motion plus local contact) produce macroscopic behavior.

The simulated space is a two-dimensional grid with periodic boundary conditions. The individual walkers are moved with a discrete-time random walk with an equal probability (20%) to either stay still or move one step in any direction that is parallel to an axis. The disease can only be transmitted when two walkers are on the exact same grid location. When infected walkers become immune, they cannot be infected again.

2. Methods

2.1 State variables and parameters

There are $N = \text{n_walk}$ walkers indexed by $i = 1..N$.

Every walker has (1) a position: (x_i, y_i) where $x_i, y_i \in 0, 1, \dots, L - 1$ and $L = \text{box_len}$, and (2) a epidemiological state: 0 for healthy, 1 for infected and 2 for immune.

Model parameters:

- `sim_len`: maximum number of time steps simulated.
- `box_len`: side length of the grid.
- `n_walk`: number of walkers.
- `n_sick`: number of initially infected walkers.
- `n_imm`: number of initially immune walkers.
- `p_sick`: per-step infection probability used by the implemented infection procedure.
- `p_heal`: per-step recovery probability for an infected walker.
- `seed`: the seed for Mersenne Twister random number generator.

2.2 Initialization

For each walker i , positions are initialized independently and (approximately) uniformly by:

- $x_i = \text{floor}(U_i L)$

- $y_i = \text{floor}(V_i L)$

where U_i , V_i are independent uniform random numbers in $[0, 1)$ produced by `grnd()` from a provided Mersenne Twister RNG module.

Health states are assigned by index:

- walkers `1..n_imm` are immune (2)
- walkers `n_imm+1..n_imm+n_sick` are infected (1)
- the remainder are healthy (0)

2.3 Movement (random walk with periodic boundaries)

At each step, each walker chooses one of five actions. The code draws `int(grnd()*5)`, which yields a value in $\{0, 1, 2, 3, 4\}$.

The implemented mapping is:

- 0: no move
- 1: $x \leftarrow x + 1$
- 2: $x \leftarrow x - 1$
- 3: $y \leftarrow y + 1$
- 4: $y \leftarrow y - 1$

Periodic boundary conditions are applied using `modulo`:

- $x \leftarrow \text{modulo}(x + dx, L)$
- $y \leftarrow \text{modulo}(y + dy, L)$

2.4 Infection rule

Infection is applied after movement and is computed in two stages: exposure detection and state update.

1. A boolean array `exposed(1..N)` is defined to `.false..` For each infected walker `i`, mark all walkers sharing its location as `exposed = .true..`
2. A random number `r` is drawn for each walker. If they are exposed and healthy they are infected if `r < p_sick`.

There are some noteworthy consequences of this exact implementation. A susceptible walker `k` that shares a site with at least one infected walker becomes infected with probability `p_sick` per time step. This probability does not increase when multiple infected walkers are co-located. Also, infection requires exact co-location; there is no neighborhood or distance-based transmission.

2.5 Recovery rule

For each infected walker `i` (`walk_cond(i) == 1`), recovery is applied independently and `setwalk_cond(i) = 2` with probability `p_heal`.

2.6 Recorded observables

For the purposes of plotting time series, at each time step the program records:

- `count(walk_cond == 1)` (infected count)
- `count(walk_cond == 2)` (immune count)

Also if required, the program can write the locations of the walkers into an `.xyz` file.

3. Implementation of the methods

3.1 Program structure

The simulation code consists of:

Module `utils` containing subroutines:

- `initialize`
- `move_walkers`
- `infect_walkers`
- `heal_walkers`

Main program `epidemic_simulation` that:

- reads command-line arguments,
- allocates arrays `walk_pos` and `walk_cond`,
- runs the time loop,
- writes output.

3.2 Data structures

The positions (x,y) for each walker are stored in an integer array `walk_pos(n_walk, 2)` and the epidemiological states $(0, 1, 2)$ are stored in a separate integer array `walk_cond(n_walk)`.

Some temporary arrays are used in the module's subroutines: `x_moves(n_walk)`, `y_moves(n_walk)` in `move_walkers` and `exposed(n_walk)` in `infect_walkers`.

3.3 External libraries and RNG

The code uses:

- Module `mtmod` containing a fortran implementation of the Mersenne Twister random number generator.
- calls to the `mtmod`'s `grnd()`, which generates a uniform random number in the range $[0, 1)$.

Therefore, compilation requires the module `mtmod` that provides `grnd()`.

3.4 Compilation

Assuming module `mtmod` and the simulator source are located in the `/src` folder, with the module in a file called `mtfort90.f90`:

```
cd ../project
make
```

The Makefile will create folders `build` and `bin`. `build` stores the compiled code and `bin` the executable.

3.5 Inputs and Usage

The executable expects 8 command-line arguments in the following order:

1. `sim_len` (integer)
2. `box_len` (integer)
3. `n_walk` (integer)
4. `n_sick` (integer)
5. `n_imm` (integer)
6. `p_sick` (real)
7. `p_heal` (real)
8. `seed` (integer)

Example:

```
cd ../project
bin/epidemic_simulation 10000 100 1000 10 0 0.5 0.01 42
```

It should be noted that the simulation terminates early if the infected count becomes zero. This is not to waste computing resources, as nothing of interest can happen when the disease dies out.

3.6 Outputs

The program writes data into a file `sim_len __ box_len __ n_walk __ n_sick __ n_imm __ p_sick __ p_heal __ seed.txt`.

Line 1: `inf,imm`.

Lines 2..`sim_len+1`: two integers per simulated step, the infected count (`inf`) and the immune count (`imm`).

A second output `epidemic.xyz` exists but is currently commented out. If enabled, it would write walker positions and states each step for visualizing their motion and the spread of the epidemic.

4. Results

4.1 Produced results

The output yields time series with `I(t)` = number of walkers with state 1 and `R(t)` = number of walkers with state 2. From these time series, one can find displays of epidemiological concepts such as extinction, where the `I(t)` reaches zero quickly, and outbreak, where `I(t)` increases initially, then declines as immunity accumulates. Because infection requires co-location, behavior depends strongly on density $\rho = N/L^2$, infectiousness `p_sick` and the infectious period `1/p_heal`.

By tweaking the parameters a bit, a sensitive combination was found, which can be seen in figure 1. “Sensitive” refers to the plot changing shape by tuning any parameter in any direction.

Epidemic simulation with parameters $L=100$, $N=1000$, $(\rho=0.1)$, $N_s=10$, $N_h=0$, $p_s=0.5$, $p_h=0.005$

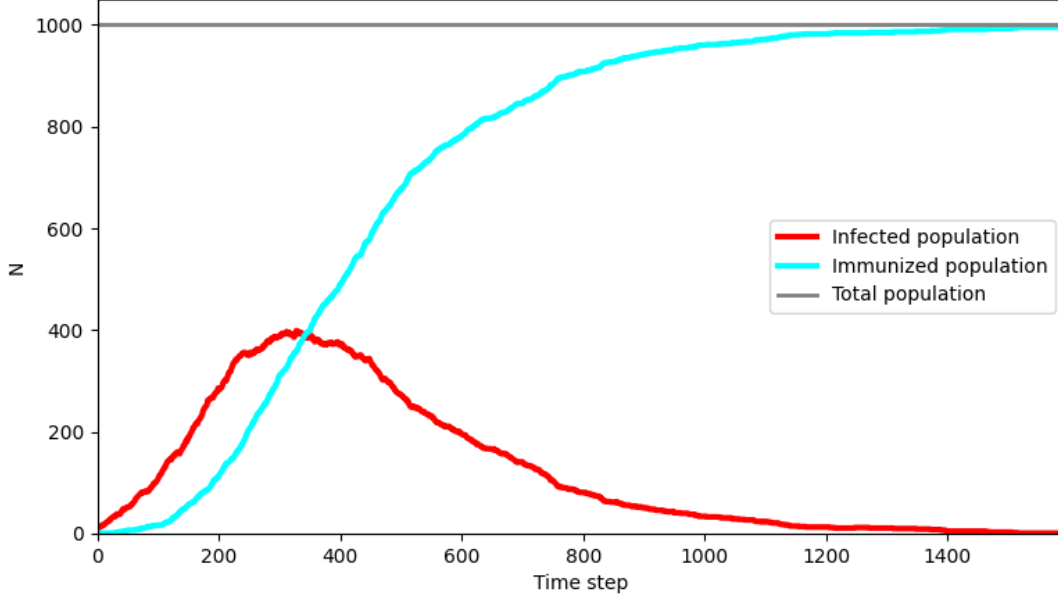


Figure 1. Simulation of an epidemic with parameters grid size of 100×100 , 1000 total amount of walkers, (corresponding to a density of 0.1), ten initially sick walkers, zero initially immune walkers, probability of transmission of 0.5 and probability of recovery of 0.005.

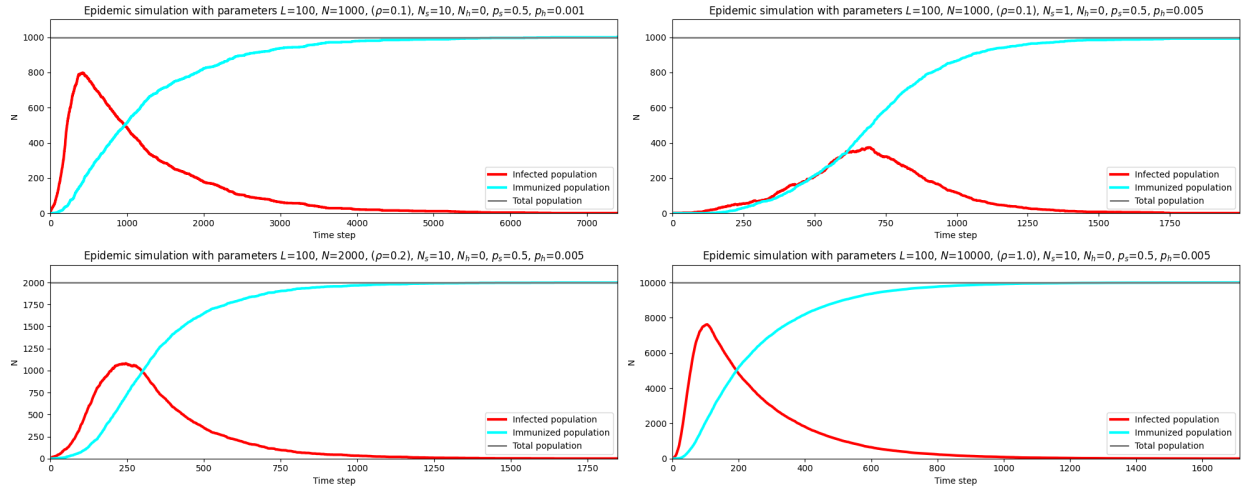


Figure 2. Four plots of simulations with varying parameters. In the top left plot the infectious period was increased five-fold, resulting in a sharp peak of infected walkers at around 400 steps that exponentially decreased for another 7000 steps until everyone had been infected. In the top right plot the simulation was started with one sick walker, resulting in a lower and lower peak than in figure 1, effectively damping the outbreak phase of the epidemic. In the bottom left plot the simulation had twice as many walkers, and consequently double the population density, which resulted in slightly sharper peak than in figure 1, where density was 0.1. In the bottom right plot the simulation had ten times as many walkers (and density) resulting in a very sharp peak around step 100.

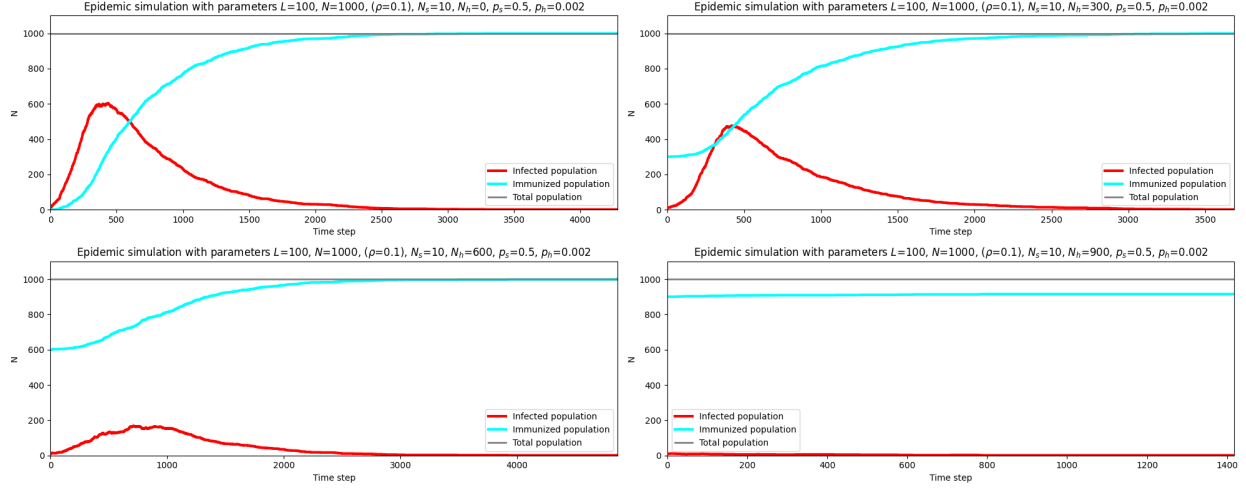


Figure 3. Four plots of simulations with varying amounts of immune walkers at the beginning, simulating disease spread in vaccinated populations. In the top right plot there are no immune walkers at the beginning, only the infectious period has been increased from figure 1 to create a sharper peak, which will allow for a better display of the damping effect of the amount of immunized population. In the top right plot 30% of the population is immune in the start, which has negligible effect on the peak of infections around step 500. In the bottom right plot 60% of the population is immune, which has had a significant damping effect on the main peak, where instead of 60% of the initially healthy population being sick at step 500, only approximately 20% are. The main peak has moved further and now resides around step 900, where about 40% of the initially healthy population are consequently sick. And finally in the bottom right plot, 90% of the population is immune at the start, resulting the disease not spreading at all, and instead dying out around step 1400.

Figure 1 shows a typical outbreak in this model. Infected walkers spread the disease only when they land on the same grid cell as a susceptible walker. With $p_{\text{sick}} = 0.5$, infection is fairly likely once contact happens, and with $p_{\text{heal}} = 0.005$ infected walkers stay infectious for a long time (about $1/p_{\text{heal}} \approx 200$ steps on average). This produces a swift rise in infections, and then a fall as more walkers become immune and there are fewer healthy ones left to infect.

Figure 2 shows how the outbreak changes when key parameters change. A longer infectious period means infected walkers have more time to spread, so the outbreak lasts longer and can infect more of the population. Starting with only one sick walker makes the start slower and more random, which lowers the peak. Higher density makes contact more frequent, so the peak happens earlier and becomes sharper; at very high density the outbreak can become very fast because walkers meet much more often.

Figure 3 shows the effect of starting with immune (vaccinated) walkers. More immunity means fewer susceptible walkers are available, so spread slows down and the peak gets smaller. With enough initial immunity, infections fail to build up and the disease dies out instead of causing a large outbreak. This is the model's version of herd immunity: reducing the susceptible pool until the effective density is low enough so that sustained spread is unlikely.

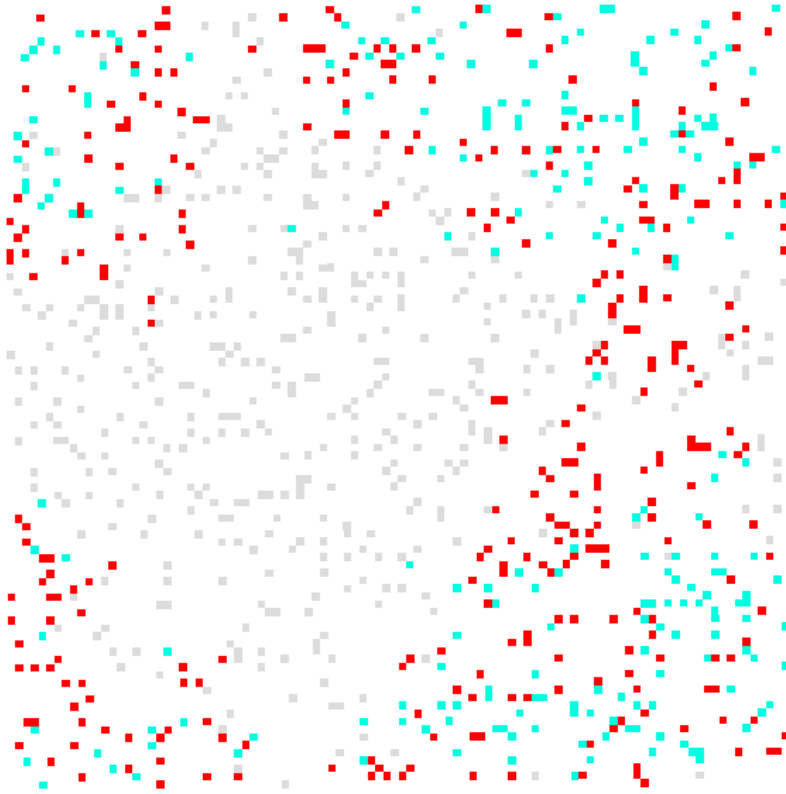


Figure 4. The simulation grid visualized with ovito. The gray squares are healthy walkers, red are sick walkers and blue are immune walkers.

4.2 Accuracy considerations

This simulator implements an unbiased random walk with a rest probability, co-location-based transmission, and an unbiased recovery with a constant probability. However, the implemented infection rule implies multiple infected walkers at one site do not increase a healthy walker’s infection probability beyond `p_sick` in a given step. This model will therefore underestimate disease transmission in crowded locations compared to a more realistic model which would simulate independent per-contact infection.

4.3 Efficiency considerations

Let $N = n_walk$.

- `move_walkers`: $O(N)$ work per time step.
- `heal_walkers`: $O(N)$ work per time step.
- `infect_walkers`: worst-case approximately $O(N^2)$ work per time step.

The infection step constructs a boolean exposure mask `exposed(1..N)`. For each infected walker `i`, the code compares that walker’s position against the full position arrays to mark all co-located walkers: `(walk_pos(:,1) == walk_pos(i,1)) and (walk_pos(:,2) == walk_pos(i,2))`

In the worst case (most walkers infected), this produces on the order of N full-length vector comparisons, i.e. $O(N^2)$ operations per step. After exposure is computed, the actual infection update is applied

once using a single masked assignment: `where (exposed .and. (walk_cond == 0) .and. (r < p_sick)) walk_cond = 1`. This final update is $O(N)$.

Overall memory usage is $O(N)$ for the state and position arrays, plus $O(N)$ temporary arrays for `exposed` and `r`.

5. Conclusions

Overall, the results show that this simulator reproduces the expected behavior of an epidemic driven by contact and recovery. Outbreak size and speed are mostly controlled by how often walkers meet (set by density and movement) and how long they stay infectious (set by recovery). Initial immunity reduces the pool of healthy walkers and can shift the system from a large outbreak to a small, delayed outbreak, and eventually to rapid die-out. This supports the idea that lowering effective contacts (or increasing immunity) is enough to stop sustained spread of a disease.

There are ways to make the simulation more realistic, though they were not instructed to be implemented. For example infection could be allowed within a local neighborhood rather than exact co-location only. Or, infection probability should depend on the number of infected present at the site, e.g. $1 - (1 - p_{\text{sick}})^m$ for m infected contacts. Maybe a latent/exposed state, a finite infectious period, or a waning immunity could be added. Also, the mobility or susceptibility could vary across walkers.